Clerk Authentication Setup Guide (SERP Server)

This documents how to wire **Clerk** into the `server` directory of the SERP project using modern **ES module** syntax. No code files are created here — this is the plan you follow when you're ready to implement.

---

1. What Is Clerk?

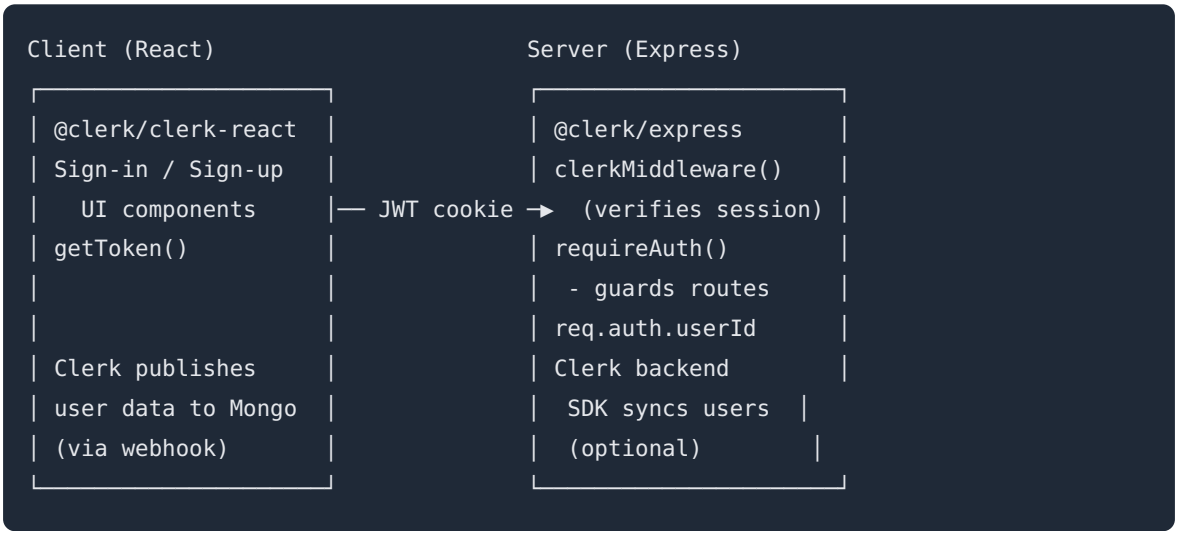
Clerk is a **full authentication-as-a-service** platform. It handles:

- Sign-up / sign-in UI (via the Clerk React SDK on the client)
- Session management and JWT issuance
- Email/SMS verification, password reset, social OAuth
- User metadata and role-based access (using `publicMetadata`)

You do **not** store passwords in your MongoDB. Clerk keeps the user records. Your server only needs to **verify** incoming session tokens on each protected request.

---

2. Architecture Overview



The flow of a request

1. User logs in on the client via Clerk-hosted UI or `<SignIn />` / `<SignUp />` components.
2. Clerk issues a **session JWT** and stores it in an HTTP-only cookie. The client can also call `getToken()` to retrieve it for explicit header-based auth.
3. Every API call from the client includes this token (automatically via cookie, or explicitly via `Authorization: Bearer <token>`).
4. On the server, `clerkMiddleware()` runs on **every** request — it reads the JWT, verifies it against Clerk's public keys, and attaches a `req.auth` object (containing `userId`, `session`, `token`, etc.).
5. On protected routes, `requireAuth()` checks that a valid session exists and either calls `next()` or returns `401 Unauthorized`.
6. For roles, check `req.auth.sessionToken?.publicSpaceId` or — more commonly — query your MongoDB user document using the Clerk `userId` to read a role from your own user record.

---

3. Prerequisites

- A Clerk account (sign up at https://clerk.com).
- A Clerk **application** created in the Clerk dashboard.
- A MongoDB cluster (already using `mongoose`).
- Node.js on the server with **ES module** support — set `"type": "module"` in `package.json` so `import`/`export` syntax works natively.

---

4. Credentials / Environment Variables Required

4.1 From the Clerk Dashboard

After you create a Clerk app, go to **API Keys** → **Development** (or **Production**) to get these values:

Variable	Prefix	Where used	Description
`NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`	`pk_`	**Client**	Public key — identifies your Clerk app to the frontend SDK. Safe to expose.
`CLERK_SECRET_KEY`	`sk_`	

Server | Secret key — used by the backend SDK to verify tokens and call the Clerk Backend API. **Never expose to the client.** |

If you plan to use **webhooks** to sync Clerk users into your MongoDB (recommended so you can attach roles and school-specific fields):

| Variable | Format | Where used | Description | |---|---|---|---| | `CLERK_WEBHOOK_SECRET` | `whsec_...` | **Server** | Signing secret for the `user.created` / `user.updated` webhook. Validates that incoming webhook events are genuinely from Clerk. |

4.2 Other server env vars (already in your existing project pattern)

| Variable | Description | |---|---| | `MONGO_URL` | MongoDB connection string (already in your `.env.example` pattern). | | `PORT` | Port to run the Express server (default `8080`). | | `CLIENT_URL` | The client origin (e.g., `http://localhost:5173`) — needed for CORS, not Clerk itself. |

4.3 .env file you will create

In `server/.env`:

```
# Clerk
CLERK_SECRET_KEY=sk_...          # from Clerk dashboard → API Keys

# MongoDB
MONGO_URL=mongodb+srv://...

# Server
PORT=8080
CLIENT_URL=http://localhost:5173

# Clerk webhook (optional — only if you sync users to MongoDB)
CLERK_WEBHOOK_SECRET=whsec_...
```

> Copy `.env.example` → `.env` and fill in real values. Add `.env` to `.gitignore` (it already is in the existing ERP project's `.gitignore`).

---

5. Server-Side Setup Steps

Step 1 — Install the Clerk Express package

```
cd server
npm install @clerk/express
```

> `@clerk/express` is the official middleware package built on top of `@clerk/backend`. It provides `clerkMiddleware` and `requireAuth` for Express. This is the **current** recommended package (v5+).

Step 2 — Set ES module mode in package.json

Update `server/package.json` from CommonJS to ES modules:

```
  "type": "commonjs"
```

becomes

```
  "type": "module"
```

> With `"type": "module"`, all `.js` files use `import` / `export` natively. You must add `.js` extensions to relative imports (e.g., `import User from '../models/User.js'`).

Step 3 — Initialise the middleware in `server.js` (or `index.js`)

You will add two things to your Express app:

```
// At the top, after importing express and mongoose
import { clerkMiddleware } from '@clerk/express';

// Inside your app setup, BEFORE any routes:
app.use(clerkMiddleware());
```

Logic: `clerkMiddleware()` intercepts every request, reads the JWT from the cookie or `Authorization` header, verifies its signature and expiry, and attaches:

- `req.auth.userId` — the Clerk user ID (e.g., `user_2x...`)
- `req.auth.sessionToken` — claims from the JWT (email, metadata, etc.)

- `req.auth.session` — the full session object

If there is no token at all (public route), `req.auth` stays `undefined` — this is fine for unauthenticated routes.

Step 4 — Create an auth middleware for role checks

You will create a new file: `server/middleware/clerkAuth.js`.

Logic:

```
import { requireAuth } from '@clerk/express';

// Protect a route — requires a valid session
const requireUser = requireAuth();

// Export for use in routes
export default requireUser;
```

Role-based access: Clerk supports roles via `publicMetadata.role` set in the Clerk dashboard, OR you can store roles in your own MongoDB user collection. The recommended approach for an ERP system:

1. Use the Clerk webhook (`user.created`) to create a matching document in your MongoDB `users` collection, storing `clerkId` , `email` , `role` (student / teacher / admin), and other profile fields.
2. On protected routes, first verify the Clerk session (`requireAuth`), then look up the MongoDB user by `clerkId` to read the `role` .

Example logic for a role-checking middleware you will write:

```
// server/middleware/clerkAuth.js (planned)
import { requireAuth } from '@clerk/express';
import User from '../models/User.js';

// 1. Verify session, then 2. attach full user from DB
export const protect = (req, res, next) => {
  requireAuth()(req, res, async () => {
    // req.auth.userId is Clerk's user ID
    const user = await User.findOne({ clerkId: req.auth.userId });
    if (!user) return res.status(404).json({ message: 'User not found' });
    req.user = user; // full user with role attached
    next();
```

```
  });
};

// Role-based wrapper
export const authorize = (...roles) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Access denied' });
  }
  next();
};
```

Step 5 — Create a User model (Mongoose)

You will create `server/models/User.js`:

```
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema({
  clerkId:  { type: String, required: true, unique: true }, // Clerk's user_xxx
  email:    { type: String, required: true, unique: true },
  name:     { type: String },
  role:     { type: String, enum: ['student', 'teacher', 'admin'], default:
'student' },
  studentId:{ type: String },   // for students
  faculty:  { type: String },   // for teachers
}, { timestamps: true });

export default mongoose.model('User', userSchema);
```

Step 6 — Set up a webhook to auto-create users

This is **critical** — every new Clerk user should be mirrored in your MongoDB.

In `server.js` (or a dedicated `routes/webhook.js`):

```
import express from 'express';

// Raw body parser needed to verify webhook signature
app.post('/api/webhooks/clerk', express.raw({ type: 'application/json' }), (req,
res) => {
  // verify the webhook signature using CLERK_WEBHOOK_SECRET
  // parse the event type
  // on "user.created": create User document in MongoDB
  // on "user.updated": update User document
```

```
  // on "user.deleted": remove User document
});
```

How to configure the webhook:

1. In the Clerk Dashboard → **Webhooks** → **Add Endpoint**.
2. URL: `https://your-api.com/api/webhooks/clerk` (use ngrok for local development).
3. Subscribe to events: `user.created`, `user.updated`, `user.deleted`.
4. Copy the **Signing Secret** → paste into `CLERK_WEBHOOK_SECRET` in `.env`.

Step 7 — Protect your routes

When you build routes for courses, assignments, documents, etc., wrap them:

```
import { protect, authorize } from '../middleware/clerkAuth.js';

router.get('/courses', protect, coursesController.getAll);
router.post('/courses', protect, authorize('teacher', 'admin'),
coursesController.create);
```

---

6. What You Do NOT Need to Build

Since Clerk handles authentication, you do **not** need:

| Not needed | Clerk handles it | |---|---| | Password hashing (bcrypt) | Clerk stores and salts passwords | | Login/register routes | `<SignIn />`, `<SignUp />` components on the client | | JWT generation/secret | Clerk issues and signs session JWTs | | Refresh token logic | Clerk's SDK manages session refresh automatically | | Email verification flow | Built-in via Clerk's dashboard settings |

---

7. Clerk Dashboard Configuration Checklist

1. **Create App** → note the Development and Production API keys.
2. **API Keys** tab → copy `CLERK_SECRET_KEY` (server) and publishable key (client).
3. **Sign-up / Sign-in** settings → enable email+password, Google, GitHub as you want.

4. **User** settings → configure which fields are required (name, email).
5. **Roles** (optional) → define `student`, `teacher`, `admin` roles in **User & Session → Roles**.
6. **Webhooks** → add endpoint URL and subscribe to `user.created`, `user.updated`, `user.deleted`.
7. **Domains** → register your client URL(s) (`http://localhost:5173` for dev, your live domain for prod).

---

8. Summary: Credentials You Must Collect

#	Credential	Key name	From where
1	Secret API Key	`CLERK_SECRET_KEY`	Clerk Dashboard → API Keys
2	Webhook Signing Secret	`CLERK_WEBHOOK_SECRET`	Clerk Dashboard → Webhooks (only if syncing users)
3	MongoDB connection	`MONGO_URL`	Your MongoDB Atlas cluster
4	Client origin	`CLIENT_URL`	Your front-end dev URL

Store all four in `server/.env` before starting the server.

---

9. Next Steps (for implementation)

When you're ready to implement, the files to create/modify are:

File	Purpose
`package.json`	Add `@clerk/express` dependency, set `"type": "module"`
`server.js`	Load `dotenv`, add `clerkMiddleware()` before routes, add CORS, register webhook route
`.env`	Add Clerk keys + MongoDB URL
`models/User.js`	Mongoose schema for local user profile + role
`middleware/clerkAuth.js`	`protect` (session verification) + `authorize` (role checking) middleware
`routes/webhook.js`	Webhook endpoint to sync users from Clerk to MongoDB
`routes/auth.js`	Optional — a `GET /me` endpoint to return the current user

The client side (in the `client/` directory) will use `@clerk/clerk-react` with `<ClerkProvider>`, `<SignedIn>`, `<SignedOut>`, `<SignIn />`, `<SignUp />`, and `useAuth()` / `useUser()` hooks. That is documented separately when we start the frontend.